Chapter 3 解答

CH03-Q01

1. `0x7f + 0x01 = 0x80`。Signed 解讀為 -128。兩個 positive operands 的和超出 127，所以 signed overflow 成立。
2. `0x80` 是 -128，`0xff` 是 -1。Bit-level 加法得 `0x7f`。Signed 為 127，mathematical sum 是 -129，兩者不同，因此 signed overflow 成立。
3. 用加法計算 `0x80 + 0xfe + 1`，結果 bit vector 仍是 `0x7f`。Mathematical result 是 $-128 - 1 = -129$，所以 signed overflow 成立。
4. 令 carry into MSB 為 c_{in}、carry out of MSB 為 c_{out}，signed overflow 也可寫成

$$ov = c_{in} \oplus c_{out}.$$

Unsigned overflow 只看 c_{out}。兩者是不同 interpretation。

CH03-Q02

初始 $A = 0000$、$Q = 1011$、$M = 1101$。每一回合先把 add result 寫成 `C:A`，再把 `(C,A,Q)` 右移。

Round	原 Q_0	Add result $C:A$	Shift 後 A	Shift 後 Q
1	1	`0:1101`	`0110`	`1101`
2	1	`1:0011`	`1001`	`1110`
3	0	`0:1001`	`0100`	`1111`
4	1	`1:0001`	`1000`	`1111`

第 2 回合的完整 9-bit shift input 是 `1:0011:1101`，右移後低 8 bits 是 `10011110`，所以 `A=1001`、`Q=1110`。C 只在該回合的加法中間存在；它隨右移進入 A，不是另外再累加第二個 carry。最後 `(A,Q)=10001111₂=143=13×11`。

固定初始multiplier為 $q < 2^n$，第 k 回合完成後 $R_k = 2^n A_k + Q_k$。歸納不變量為

$$R_k = \lfloor q/2^k \rfloor + M(q \bmod 2^k)2^{n-k}.$$

基底 $A_0 = 0, Q_0 = q$ 成立。$k < n$ 時第二項為偶數，所以舊 Q_k 的最低位為原 q 的第 k 位 q_k。加法與完整carry右移實作

$$R_{k+1} = \lfloor (R_k + q_k M 2^n)/2 \rfloor = \lfloor q/2^{k+1} \rfloor + M(q \bmod 2^{k+1})2^{n-k-1},$$

其中使用 $q \bmod 2^{k+1} = q \bmod 2^k + q_k 2^k$。因此 $R_n = Mq$，A保存乘積高半部，Q保存低半部。舊Q0已在本回合處理並丟棄，下回合decision bit是移入新Q0的舊Q1。數例的 $R_0, \ldots, R_4$ 為11、109、158、79、143，與每列寄存器一致。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 3 解答 （續）

32-bit serial latency 是

$$T_{\text{serial}} = 3 + 32(1+1) + 2 = 69 \text{ cycles}.$$

兩個 generation 若可重疊，steady-state initiation interval 是 4 cycles，因此 throughput 是 $1/4 = 0.25$ multiplication/cycle，單筆 latency 仍是 69 cycles。Pipeline 需要把 partial product 的 register 與 control state 分離、提供 collision detection，並在 context/operand 間保存每 stage 的狀態；不能只複製 ALU。

CH03-Q03

初始 $A = 00000$、$Q = 1101$、$M = 00101$。

Round	合併左移後 A, Q	$A - M$	Decision 後
1	`00001,1010`	-4	restore；`00001,1010`
2	`00011,0100`	-2	restore；`00011,0100`
3	`00110,1000`	+1	keep；`00001,1001`
4	`00011,0010`	-2	restore；`00011,0010`

1. Trace 如表。
2. Final $A = 00011_2 = 3$、$Q = 0010_2 = 2$。故 $13 \div 5$ 得 quotient 2、remainder 3。
3. $5 \times 2 + 3 = 10 + 3 = 13$，且 $0 \le R < 5$，成立。
4. `-2147483648 / -1` 的 exact quotient $+2147483648$ 超出 signed 32-bit 上界 2147483647，是數學上的溢位邊界；在本章傳統 MIPS32 不會由 `div` 自動產生 overflow trap，HI/LO 結果不可依賴，必須先用軟體檢查。其他 ISA 的例外政策需另行確認。

CH03-Q04

1. $12.5 = 1100.1_2 = 1.1001_2 \times 2^3$。Sign bit 為 1，biased exponent 為 $3 + 127 = 130 = 10000010_2$，fraction 是 `10010000000000000000000`。

```
1 10000010 10010000000000000000000
```

分組 hex 得 `0xc1480000`。

2. `0x3f000000` 的 sign 為 0，exponent field 為 `01111110_2=126`，fraction 全 0。Value 是

$$1.0 \times 2^{126-127} = 0.5.$$

3. Smallest positive normal：

Chapter 3 解答 （續）

- exponent field 為 `00000001`，biased exponent 1，unbiased -126。
- fraction 全 0。
- value 2^{-126}。

Smallest positive subnormal：

- exponent field 為 `00000000`。
- fraction 只有 LSB 為 1。
- value $2^{-23}2^{-126} = 2^{-149}$。

4. `0x7f800000` 的 exponent field 全 1、fraction 為 0，是 positive infinity。`0x7fc00001` 的 exponent field 全 1、fraction 非 0，是 NaN；leading fraction bit 使其成為 quiet NaN。

CH03-Q05

1. Binary32 的有效精度為24 bits（含hidden 1）；在1附近且不小於1時，相鄰值間距為 2^{-23}。所以 exact $1 + 2^{-24}$ 的兩個相鄰 candidates 是

$$1.0 \quad 與 \quad 1 + 2^{-23}.$$

2. Exact result 正好位於兩者中點。Round-to-nearest-even 選擇最低保留 significand bit 為 0 的 candidate。1.0 的 significand bits 全為 0。
3. Result 是 `0x3f800000`，value 1.0；這次運算應標記 inexact；旗標記錄發生不精確捨入，不儲存誤差數值，是否 trap 另由浮點環境控制。
4. 若用更高 precision 保留 `1+2^-24`，該 intermediate value 不會因 target binary32 precision 而先消失。之後做 `-1` 可留下 2^{-24}。最後寫回 binary32 是 exact representable。所以 intermediate precision 與 rounding point 會改變結果。

CH03-Q06

1. $1.5 = 1.1_2 \times 2^0$。2^{-126} 是 binary32 的 smallest positive normal，significand 1.0_2、unbiased exponent -126。
2. Significand product 為

$$1.1_2 \times 1.0_2 = 1.1_2.$$

Exact exponent 是 -126。

3. Product 已在 normalized form，不需要 left shift。
4. 欄位儲存 biased exponent。加法後應在 unbiased arithmetic 中計算 -126，再依 target bias 加 127；直接相加欄位會把兩份 bias 混入。同時 significand carry 可能使 exponent 再增加 1，之後的 rounding 也可能再 normalize。

Chapter 3 解答 （續）

CH03-Q07

1. 2^{24} 的 binary32 ULP 是 2。$2^{24}+1$ 正好是 2^{24} 與 $2^{24}+2$ 的中點。最低保留 bit 的 even candidate 是 2^{24}，所以第一加法的 binary32 result 是 2^{24}。再減 2^{24} 得 0。
2. $2^{24}-2^{24}=0$，加 1 得 1。
3. Association 失效發生在第一次 rounding。Exact associativity 應允許先加後加的 grouping 相同，但第一個 grouping 已把 +1 丟到 precision 外。
4. Extended-precision model 可保留 $1+2^{-24}$，減 1 後得到 2^{-24}；binary32 per-operation model 在第一次加法得到 1，最後為 0。結論建立在「x87 保留該精確 value」與 control mode 的假設上，並非所有環境都如此。

CH03-Q08

1. Sequential latency：

$$3+32(1)+2=37\ \text{cycles}.$$

Radix-4 latency：

$$6+16(1.5)=30\ \text{cycles}.$$

2. 在兩者都可每 8 cycles 接受新 operation 的假設下，穩定 throughput 都是

$$\frac{1}{8}=0.125\ \text{operations/cycle}$$

未給 clock frequency，不能換算為每秒 operations。以每 1000 cycles 計，兩者都是 125 operations。

3. Latency 決定單一 dependent operation 從 input 到 output 的時間，影響 recurrence 與 critical path。Throughput 決定多個 independent operations 的穩定速率。若 application 有很多 independent multiplies，pipeline throughput 可能主導；若下一 operation 依賴前一結果，latency 主導。
4. Divider 的 16 次 radix-4 iterations 仍需 quotient-digit selection、normalization、correction、sign handling、divide-by-zero 與 remainder normalization。不同 operand patterns 可能造成不同 iteration count 或 correction path，unless the timing is specified as input-independent。若存在 data-dependent early termination，平均值也需要 workload distribution。

Chapter 3 解答 （續）

CH03-Q09

1. `1.0` 的可保留 significand 是 `1.00000000000000000000000`。加入 2^{-24} 後，被丟棄的片段是 `100`，即 $G=1, R=0, S=0$。Tie 的兩個 candidates 是 `1.0` 與 $1+2^{-23}$；$L=0$ 的 `1.0` 是 even，所以 result 是 `0x3f800000`，並設 inexact。
2. $3\cdot 2^{-25} = 2^{-24}+2^{-25}$。相對於 retained 的 2^{-23} unit，discarded bits 是 `110`，即 $G=1, R=1, S=0$。它大於一半，因此加一個 retained unit，result 是 $1+2^{-23}$（fraction LSB 為 1）。
3. 2^{-151} 小於 2^{-149} 的一半，朝向 $+0$ 與 2^{-149} 的最近候選是 zero；它 tiny 且 inexact，因此 set underflow 與 inexact。2^{-150} 正好 halfway；even candidate 是 $+0$，所以 result 為 `0x00000000`，同樣 set underflow 與 inexact。這裡不把 underflow 誤寫成「只有 exponent arithmetic 進位」。
4. Maximum finite binary32 是 `0x7f7fffff`。加自身後 exact result 遠大於 max；default RNE 產生 `0x7f800000`（$+\infty$），set overflow 與 inexact。`0×∞` 屬於 invalid operation，default quiet result 是 NaN，set invalid；不可只因 multiplication of zero 就回傳 zero。

CH03-Q10

1. $1+2^{-23}$ 的 exponent field 是 127、fraction 的 LSB 是 1，所以 `a=b=0x3f800001`。$1+2^{-22}$ 的 fraction 是 `0x000002`，加負號後 `c=0xbf800002`。
2. Exact product 是

$$ab = (1+2^{-23})^2 = 1+2^{-22}+2^{-46}.$$

在 exponent 0，binary32 的相鄰 spacing 是 2^{-23}；2^{-46} 遠小於 half-ULP，所以 `mul.s` 把 product round 成 $1+2^{-22}$。再與 $c=-(1+2^{-22})$ 相加，exact zero 且兩 operand 同 magnitude，result 為 `+0`（`0x00000000`）。這一步的 addition 不再需要保留第一次 rounding 丟掉的 2^{-46}。 3. FMA 將 $ab+c$ 保持為一個 exact intermediate：

$$(1+2^{-22}+2^{-46}) - (1+2^{-22}) = 2^{-46}.$$

2^{-46} 是 binary32 normal，unbiased exponent 是 -46，biased exponent 是 $127-46=81=0x51$，sign 與 fraction 全 0，因此 bits 是 `0x28800000`。這和分開指令得到 zero 不同，差異來自 intermediate rounding，不是 FMA instruction 任意改寫 floating-point algebra。 4. $x=2^{24}$ 的 ULP 是 2。`x+1` 是 x 與 $x+2$ 的 tie，RNE 選 even 的 x，因此 `(x+1)-x=0`；先 `x-x=0` 再加 1 得到 1。改變的是 `x+1` 的 rounding point。Binary32 每一輪運算只保留 24 個 significand bits；exact real arithmetic 的 associativity 不能跨越每次 binary32 rounding 自動繼承。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 3 解答 （續）

原創驗算程式附錄: ieee754_explorer.py

```
import math
import struct


def bits_to_f32(bits: int) -> float:
    return struct.unpack(">f", struct.pack(">I", bits))[0]


def f32_to_bits(value: float) -> int:
    try:
        return struct.unpack(">I", struct.pack(">f", value))[0]
    except OverflowError:
        return 0xFF800000 if value < 0 else 0x7F800000


def decode_f32(bits: int) -> dict[str, int | float | str]:
    sign = bits >> 31
    exponent = (bits >> 23) & 0xFF
    fraction = bits & 0x7FFFFF
    if exponent == 0xFF:
        kind = "infinity" if fraction == 0 else "NaN"
    elif exponent == 0:
        kind = "zero" if fraction == 0 else "subnormal"
    else:
        kind = "normal"
    return {
        "sign": sign,
        "exponent_field": exponent,
        "fraction_field": fraction,
        "kind": kind,
        "value": bits_to_f32(bits),
    }


def round_nearest_even(guard: int, round_bit: int, sticky: int, lsb: int) -> bool:
    """Return whether the retained significand increments."""
    return bool(guard and (round_bit or sticky or lsb))


def main() -> None:
    assert f32_to_bits(-12.5) == 0xC1480000
    assert decode_f32(0x3F000000)["value"] == 0.5
    assert decode_f32(0x00000001)["kind"] == "subnormal"
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 3 解答 （續）

```
    assert decode_f32(0x00000001)["value"] == math.ldexp(1.0, -149)
    assert decode_f32(0x00800000)["value"] == math.ldexp(1.0, -126)
    assert decode_f32(0x7F800000)["kind"] == "infinity"
    assert decode_f32(0x7FC00001)["kind"] == "NaN"

    rounded_once = bits_to_f32(f32_to_bits(1.0 + math.ldexp(1.0, -24)))
    assert rounded_once == 1.0

    x = float(2**24)
    left_grouping = (bits_to_f32(f32_to_bits(x + 1.0)) - x)
    right_grouping = (x - x) + 1.0
    assert left_grouping == 0.0
    assert right_grouping == 1.0
    assert left_grouping != right_grouping

    assert round_nearest_even(guard=1, round_bit=0, sticky=0, lsb=0) is False
    assert round_nearest_even(guard=1, round_bit=1, sticky=0, lsb=0) is True

    a = bits_to_f32(0x3F800001)  # 1 + 2^-23
    c = bits_to_f32(0xBF800002)  # -(1 + 2^-22)
    separate_product = bits_to_f32(f32_to_bits(a * a))
    assert f32_to_bits(separate_product) == 0x3F800002
    assert separate_product + c == 0.0
    assert f32_to_bits(math.ldexp(1.0, -46)) == 0x28800000

    assert f32_to_bits(math.ldexp(1.0, -150)) == 0x00000000
    assert f32_to_bits(bits_to_f32(0x7F7FFFFF) * 2.0) == 0x7F800000

    print(f"-12.5 bits: 0x{f32_to_bits(-12.5):08x}")
    print(f"smallest subnormal: {decode_f32(0x00000001)['value']:.17e}")
    print(f"1 + 2^-24 rounds to: {rounded_once}")
    print(f"grouping comparison: {left_grouping} != {right_grouping}")
    print(f"separate mul+add: 0x{f32_to_bits(separate_product + c):08x}")
    print(f"fused exact result: 0x{f32_to_bits(math.ldexp(1.0, -46)):08x}")
    print("IEEE-754 checks: PASS")


if __name__ == "__main__":
    main()
```

參考資料與署名

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study